

Day-73-Pandas Fundamentals — DataFrames

1. Day Number

Day 73

2. Topic Name

Pandas Series and DataFrame Basics

Today you will learn the basic Pandas structures used to work with data arranged in labeled rows and columns.

3. Connection

Yesterday, you used **NumPy arrays** to store numbers and perform calculations.

Today, you will move one step closer to real-world data analysis by using **Pandas**.

NumPy is excellent for arrays of values. Pandas makes it easier to work with data that looks like a table:

Product	Price	Stock
Laptop	50000	5
Mouse	800	20
Keyboard	1500	10

Instead of remembering what each position means, Pandas lets you give columns meaningful names such as Product, Price, and Stock.

4. Important Topics

Pandas

Pandas is a Python library commonly used for:

- organizing data
- reading datasets
- selecting rows and columns
- cleaning data
- analyzing data

It is commonly imported using:

```
import pandas as pd
```

pd is just a short name for Pandas.

If Pandas is not installed:

```
pip install pandas
```

Series

A **Series** is similar to one column of data.

For example:

```
Price
50000
800
1500
```

You can think of a Series as:

```
One labeled column of values
```

DataFrame

A **DataFrame** is a table containing rows and columns.

Example:

Product	Price	Stock
Laptop	50000	5
Mouse	800	20
Keyboard	1500	10

A DataFrame can contain several Series.

Rows

A **row** usually represents one record.

For example:

Laptop	50000	5
--------	-------	---

This row describes one product.

Columns

A **column** usually represents one type of information.

For example:

```
Price
50000
800
1500
```

Column Names

Column names describe what the values represent.

Examples:

```
Product
Price
Stock
```

Good column names make datasets easier to understand.

5. Foundational Notes

Think of a DataFrame like a small Excel worksheet inside Python.

Suppose a shop has this information:

Product	Price	Stock
Laptop	50000	5
Mouse	800	20
Keyboard	1500	10

There are:

```
3 rows
3 columns
```

The columns are:

```
Product
Price
Stock
```

Each row represents one product.

A DataFrame can be created from several Python structures. One beginner-friendly approach is using a **dictionary containing lists**.

Conceptually:

```
data = {
    "Product": [...],
    "Price": [...],
    "Stock": [...]
}
```

Then Pandas can convert that dictionary into a DataFrame.

6. NumPy Array vs Pandas DataFrame

A NumPy array mainly focuses on **values and positions**.

Example:

```
prices = np.array([50000, 800, 1500])
```

You might access a value using its position:

```
prices[0]
```

A Pandas DataFrame adds meaningful labels.

Product	Price	Stock
Laptop	50000	5
Mouse	800	20
Keyboard	1500	10

So the simple difference is:

NumPy Array	Pandas DataFrame
Mainly values	Table-like data
Usually position-based	Has labeled columns
Great for numerical calculations	Great for structured datasets
Similar to a mathematical array	Similar to an Excel table

For now, remember:

NumPy = arrays of values
Pandas DataFrame = labeled rows and columns

Pandas actually works closely with NumPy internally.

7. Easy Example

Imagine we want to store student information.

Our data could conceptually look like:

```
student_data = {  
    "Name": ["Asha", "Ravi", "Neha"],  
    "Score": [85, 72, 91]  
}
```

After converting it into a DataFrame, it would look approximately like:

	Name	Score
0	Asha	85
1	Ravi	72
2	Neha	91

Notice the numbers:

0
1
2

These are row labels automatically created by Pandas.

The columns are:

Name
Score

If you select only the Score column, Pandas gives you something similar to:

0	85
1	72
2	91

That single column is usually a **Pandas Series**.

8. Problem Statement

Create a small Pandas DataFrame containing:

Product Name
Price
Stock

Use approximately three products.

For example, your data might represent:

Laptop
Mouse
Keyboard

Your program should:

1. Import Pandas.
2. Create product data.
3. Convert the data into a DataFrame.

4. Print the complete DataFrame.
5. Select the Price column.
6. Print that selected column.

Do not use advanced indexing techniques yet.

9. Concepts Used

For this problem, you will practice:

```
importing Pandas
Python dictionaries
Python lists
DataFrame creation
rows
columns
column names
selecting one column
print()
```

The important new concept is:

```
pd.DataFrame(...)
```

This tells Pandas to create a DataFrame from your data.

You will also learn the basic idea of selecting a column using its name.

Conceptually:

```
dataframe["column_name"]
```

10. Thought Process

Before writing the program, think about the data itself.

You need three pieces of information about every product:

```
Product
Price
Stock
```

So these naturally become your **columns**.

Then decide on a few products.

For example:

Product	Price	Stock
Laptop	50000	5
Mouse	800	20
Keyboard	1500	10

Now think about how Python can represent those columns.

You already know dictionaries and lists.

A dictionary can give each list a meaningful name:

```
Product -> list of product names
Price   -> list of prices
Stock   -> list of stock amounts
```

Pandas can then turn this structured dictionary into a DataFrame.

Finally, select one column using its column name.

11. Beginner-Friendly Pseudocode

```
START

Import Pandas

Create a dictionary called product_data

Inside the dictionary:
    create a Product column
    create a Price column
    create a Stock column

Create a DataFrame from product_data

Print the complete DataFrame

Select the Price column

Print the Price column

END
```

Notice that the pseudocode describes the **logic** without giving you the complete Python solution.

12. Suggested Solving Approach — Pandas Approach

Use this general flow:

```
Python data
  ↓
Dictionary containing lists
  ↓
Pandas DataFrame
  ↓
Print DataFrame
  ↓
Select one column
  ↓
Print selected column
```

A useful mental model is:

```
Dictionary keys → column names
Dictionary lists → column values
```

For example:

```
Product → Laptop, Mouse, Keyboard
Price   → 50000, 800, 1500
Stock   → 5, 20, 10
```

Pandas combines these into one table.

13. Easy Edge Cases

Empty DataFrame

Sometimes there may be no data.

Conceptually:

```
empty_data = {}
```

A DataFrame created without useful data may appear as:

```
Empty DataFrame
Columns: []
Index: []
```

An empty DataFrame is valid. It simply contains no records.

Zero Stock

A product can exist even when its stock is zero.

Example:

Product	Price	Stock
Laptop	50000	5
Mouse	800	0
Keyboard	1500	10

0 stock does **not** mean the row should disappear.

It means:

The product exists,
but currently no units are available.

So zero is valid data.

14. Expected Output

Your exact values can be different, but the complete DataFrame might look approximately like:

	Product	Price	Stock
0	Laptop	50000	5
1	Mouse	800	20
2	Keyboard	1500	10

Then printing only the price column might look like:

0	50000
1	800
2	1500
Name: Price, dtype: int64	

Do not worry too much about this part yet:

dtype: int64

It simply tells you that Pandas recognized the column as containing integer numbers.

Also notice:

Name: Price

This appears because selecting one column usually produces a **Series**.

15. Hint Only

Start with:

```
import pandas as pd
```

Then think about creating a dictionary with three keys:

```
Product
Price
Stock
```

Each key should contain a list with the same number of items.

To turn your dictionary into a DataFrame, investigate:

```
pd.DataFrame(...)
```

To access one column, remember that a DataFrame can use a column name inside square brackets:

```
dataframe["..."]
```

Your challenge is to replace the missing pieces yourself.

Key idea for Day 73

```
NumPy array
  ↓
values arranged in an array

Pandas Series
  ↓
one labeled sequence / column

Pandas DataFrame
  ↓
multiple labeled columns forming a table
```

For today, focus only on **creating a DataFrame, understanding rows and columns, and selecting a single column**. Advanced indexing can come later.